Given these sample points:

[0.31940389 0.76295937]
[0.57432921 0.8798981]
[0.73102363 0.73299988]
[0.84035342 0.26473161]
[0.65011406 0.68152635]
[0.41043714 0.1475543]
[0.31269116 0.07872278]
[0.68341817 0.86105746]
[0.08250725 0.40348751]
[0.88388983 0.58225397]
[0.908656 0.808513]
[0.43 0.5]
[0.611853 0.139494]
[0.889128 0.578807]
[0.4 0.7]
[0.7 0.45]
[0.807957 0.767765]

And these results:

1.3226770395454077e-79
1.0330782375230975e-46
7.710875114502849e-16
3.341771007676023e-124
-0.0036060626443634764
-2.1592490357331095e-54
-2.0890932702320842e-91
2.5350011535584046e-40
3.6067711901420254e-81
6.229856468168659e-48
2.358757555918235e-78
-4.336733960179099e-05
2.4247688654824617e-82
5.479582068424992e-50
2.653959939729813e-40
-2.5135190519902422e-26
1.3771514819813648e-36

Where should I sample next to obtain the maximum result ?

Use Temperate = 0.1, Top-k = 5, Top-p = 5 in the prediction model

Your results are overwhelmingly near zero (or tiny positive/negative noise), except for a few **relatively large (less negative)** values:

- **[0.43, 0.5] → -4.33e-05** (best so far)
- **[0.6501, 0.6815] → -0.0036** (worse but still "large")
- **[0.7, 0.45] → -2.51e-26** (close to zero, but still worse than -4e-05)

Everything else is essentially ≈ 0.

Key Insight

Your function appears to:

- Peak near **[0.43, 0.5]**
- Drop off *very sharply* (values collapse to ~1e-40 or smaller almost everywhere)

This strongly suggests:

- A **very narrow maximum**
- Likely centered near **(0.43, 0.5)**

Where to sample next

Since you're using low temperature (0.1) and small top-k/top-p (i.e., exploitative search), you should **refine locally around the best point**.

Recommended next samples (tight local search)

Focus on a small neighborhood:

```csharp
[0.42, 0.50]
[0.44, 0.50]
[0.43, 0.49]
[0.43, 0.51]
[0.425, 0.495]
[0.435, 0.505]
```

If you want slightly broader exploration (still safe)

```csharp
[0.41, 0.50]
[0.45, 0.50]
[0.43, 0.48]
[0.43, 0.52]
```

Why this works

- Your data strongly indicates a **sharp spike**
- Global exploration is already done (points are scattered)
- Now you need **local exploitation**

This is exactly what methods like:

- Bayesian Optimization
- Expected Improvement
- Gaussian Processes

would do here.

Bottom line

👉 **Next best sample:**

```scss
[0.43, 0.50] ± small perturbations (~0.01 or less)
```

If you want a single best guess:

```csharp
[0.435, 0.505]
```

If you want, I can:

- Fit a surrogate model to your data
- Estimate the peak more precisely
- Or suggest a full optimization strategy (Bayesian / gradient-free)

Given these sample points:

[0.31940389 0.76295937]
[0.57432921 0.8798981]
[0.73102363 0.73299988]
[0.84035342 0.26473161]
[0.65011406 0.68152635]
[0.41043714 0.1475543]
[0.31269116 0.07872278]
[0.68341817 0.86105746]
[0.08250725 0.40348751]
[0.88388983 0.58225397]
[0.908656 0.808513]
[0.43 0.5]
[0.611853 0.139494]
[0.889128 0.578807]
[0.4 0.7]
[0.7 0.45]
[0.807957 0.767765]

And these results:

1.3226770395454077e-79
1.0330782375230975e-46
7.710875114502849e-16
3.341771007676023e-124
-0.0036060626443634764
-2.1592490357331095e-54
-2.0890932702320842e-91
2.5350011535584046e-40
3.6067711901420254e-81
6.229856468168659e-48
2.358757555918235e-78
-4.336733960179099e-05
2.4247688654824617e-82
5.479582068424992e-50
2.653959939729813e-40
-2.5135190519902422e-26
1.3771514819813648e-36

A small positive number is bigger than a large negative number

Where should I sample next to obtain the maximum result ?

Use Temperate = 0.1, Top-k = 5, Top-p = 5 in the prediction model

You're absolutely right: **any small positive value beats any negative value**, so we're really looking for where the function is **positive and as large as possible**.

Step 1: Identify the best observed regions

Let's focus on the **largest positive values** (ignoring tiny ~1e-80 noise):

- **[0.7310, 0.7330] → 7.7e-16** ✅ (best positive)
- **[0.4, 0.7] → 2.65e-40**
- **[0.6834, 0.8611] → 2.53e-40**

Everything else is either:

- Extremely close to zero (≈ numerical noise), or
- Negative (worse)

👉 So the **only meaningful signal** is around:

```scss
(0.73, 0.73)
```

Step 2: Compare with misleading region

The point:

```csharp
[0.43, 0.5] → -4.33e-05
```

is actually **bad**, because:

- It is **negative**
- Even tiny positives (like 1e-40) are better

So we should **avoid that region entirely**

Step 3: Strategy (given low temperature, top-k=5)

Your model settings imply:

- Strong **exploitation**
- Focus on **top candidates only**

So we:

1. Take the best point: **(0.73, 0.73)**
2. Sample **locally around it**

Step 4: Recommended next samples

Primary (tight exploitation)

```csharp
[0.72, 0.73]
[0.74, 0.73]
[0.73, 0.72]
[0.73, 0.74]
[0.735, 0.735]
```

Secondary (slight expansion)

```csharp
[0.71, 0.73]
[0.75, 0.73]
[0.73, 0.71]
[0.73, 0.75]
```

Step 5: Single best next guess

If you must pick **one point**:

```css
👉 [0.735, 0.735]
```

Intuition

- The function behaves like a **sharp positive spike**
- Most of the domain ≈ 0 or negative
- Only one region shows **meaningful positive signal**
- So the maximum is very likely:
  - **near (0.73, 0.73)**
  - possibly slightly offset

Final Answer

👉 Sample next at:

```csharp
[0.735, 0.735]
```

and explore a **small neighborhood around (0.73, 0.73)**.